

Semantic Wave Function Collapse (SWFC)

Architectural Standard for Just-In-Time Procedural Reality

- **Author:** Patrick Richardson
- **Date:** December 19, 2025
- **Status:** Architectural Standard v1.0
- **Version:** 1.0 (Release)

1. Executive Summary

Traditional game engines treat **Narrative** (Flavor Text, Visuals) and **Systems** (Stats, Mechanics) as separate domains. This separation leads to *Ludonarrative Dissonance*, where a sword described as "cursed" behaves identically to a standard iron blade.

Semantic Wave Function Collapse (SWFC) is a unified architecture that resolves this issue by treating narrative and mechanics as **two views of the same canonical state**, revealed progressively through observation.

Unlike pure generative AI, SWFC does not grant the **Large Language Model (LLM)** authority over the game state. Instead, the LLM acts as a **Semantic Proposer** within a strict, validated constraint-satisfaction loop. This ensures that while the content is infinite, the logic remains **rigid, deterministic, and persistent**.

2. Problem Statement: The Three Traps

Current procedural architectures suffer from three specific disconnects that prevent true immersion. SWFC is explicitly designed to solve these architectural failures.

2.1 The Semantic Gap (The "Math vs. Meaning" Trap)

Standard procedural algorithms, such as Perlin Noise or RNG loot tables, are semantically blind.

- *The Failure:* A generic "Level 5 Loot Table" drops items that mathematically fit the player's level but thematically contradict the context.
- *Example:* A Fire Elemental drops a "Frost Wand" simply because the RNG rolled a valid ID, breaking immersion.

2.2 The Hallucination Trap (The "Dream Logic" Flaw)

Pure Generative AI lacks state rigidity. Without a canonical anchor, LLMs suffer from object impermanence.

- *The Failure*: An NPC described as "mortally wounded" in Turn 1 might appear "perfectly healthy" in Turn 2 because the model context window shifted or the model hallucinated a repair.
- *Result*: The world feels like a dream: fluid and reacting to the immediate prompt, but lacking causal history.

2.3 The Latency Trap (Static Fidelity)

To ensure coherence, traditional games must pre-simulate the entire world (e.g., Dwarf Fortress) or fully instantiate objects upon creation.

- *The Failure*: The game must store the exact stats of a sword the moment it drops, even if the player never picks it up. This wastes resources on unobserved reality and prevents the world from reacting dynamically to player actions.
- *Result*: High CPU/Memory overhead for low-interaction fidelity.

2.4 Note on Nomenclature

Note: While this architecture adopts the term "Wave Function Collapse" to describe the reduction of possibility space via constraint satisfaction, it is distinct from the pixel-adjacency algorithm popularized by Maxim Gumin (2016). SWFC replaces rigid adjacency matrices with **Semantic Reasoning** as the constraint solver. It is an evolution of the philosophy, not a direct implementation of the bitmap algorithm.

3. Core Definitions & Invariants

3.1 Core Terms

Latent Entity (Superposition):

An entity that exists only as an opaque handle (ID) and a set of Constraints. It has no fully realized components (e.g., no stats, no mesh), only potential.

Observation:

The event where a Player or System queries a property of a Latent Entity, triggering the generation loop.

Observation Scope:

Defined strictly by the Interaction Horizon:

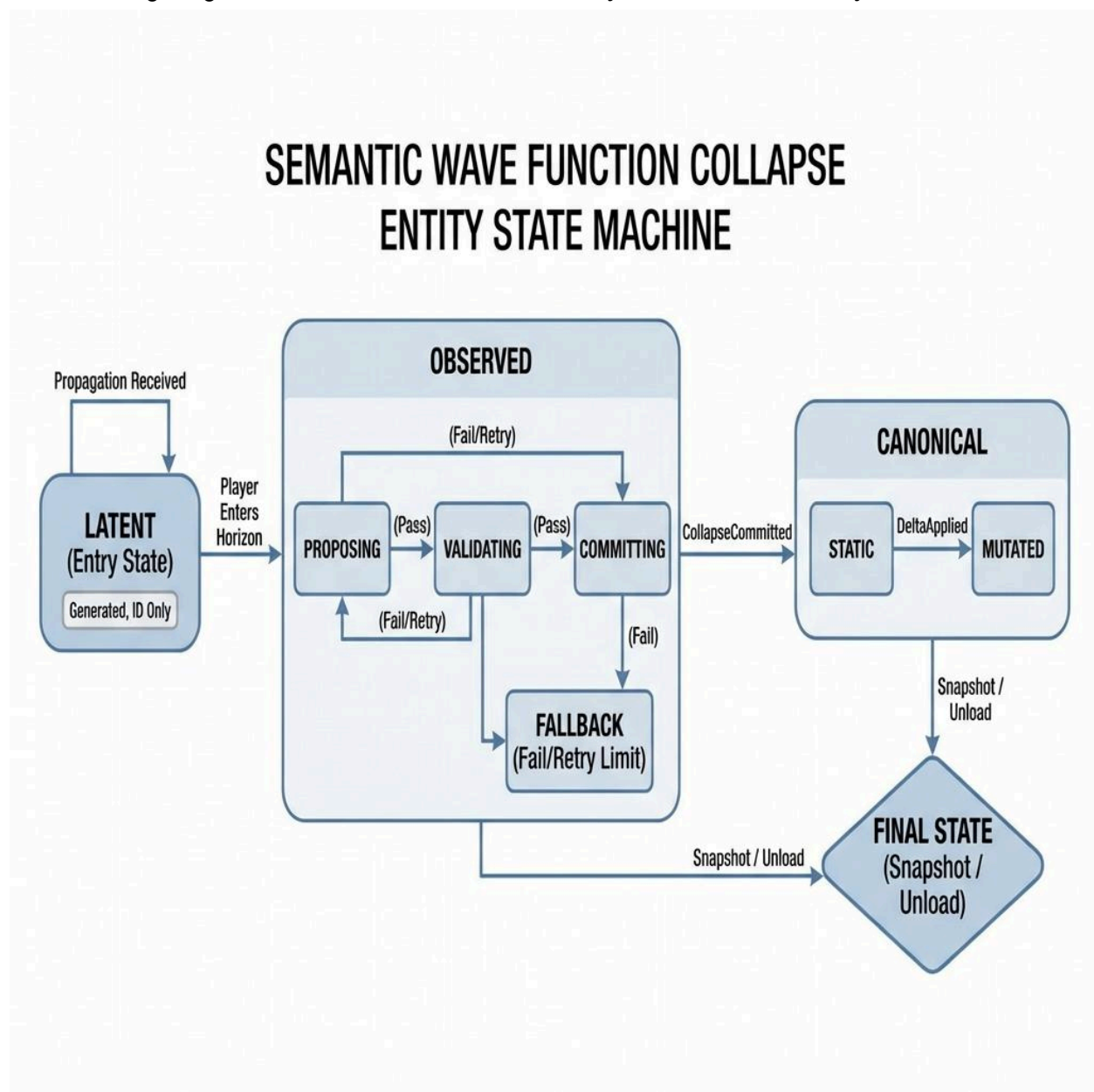
- **Visual Horizon**: Camera Frustum + Buffer
- **Logical Horizon**: Graph Distance (e.g., Depth 2 neighbors)

Canonical State:

A Materialized View (Projection) derived from the Event Log. It serves as the fast runtime database but is not the ultimate source of truth.

3.2 Entity State Machine

The following diagram illustrates the immutable lifecycle of an SWFC entity.



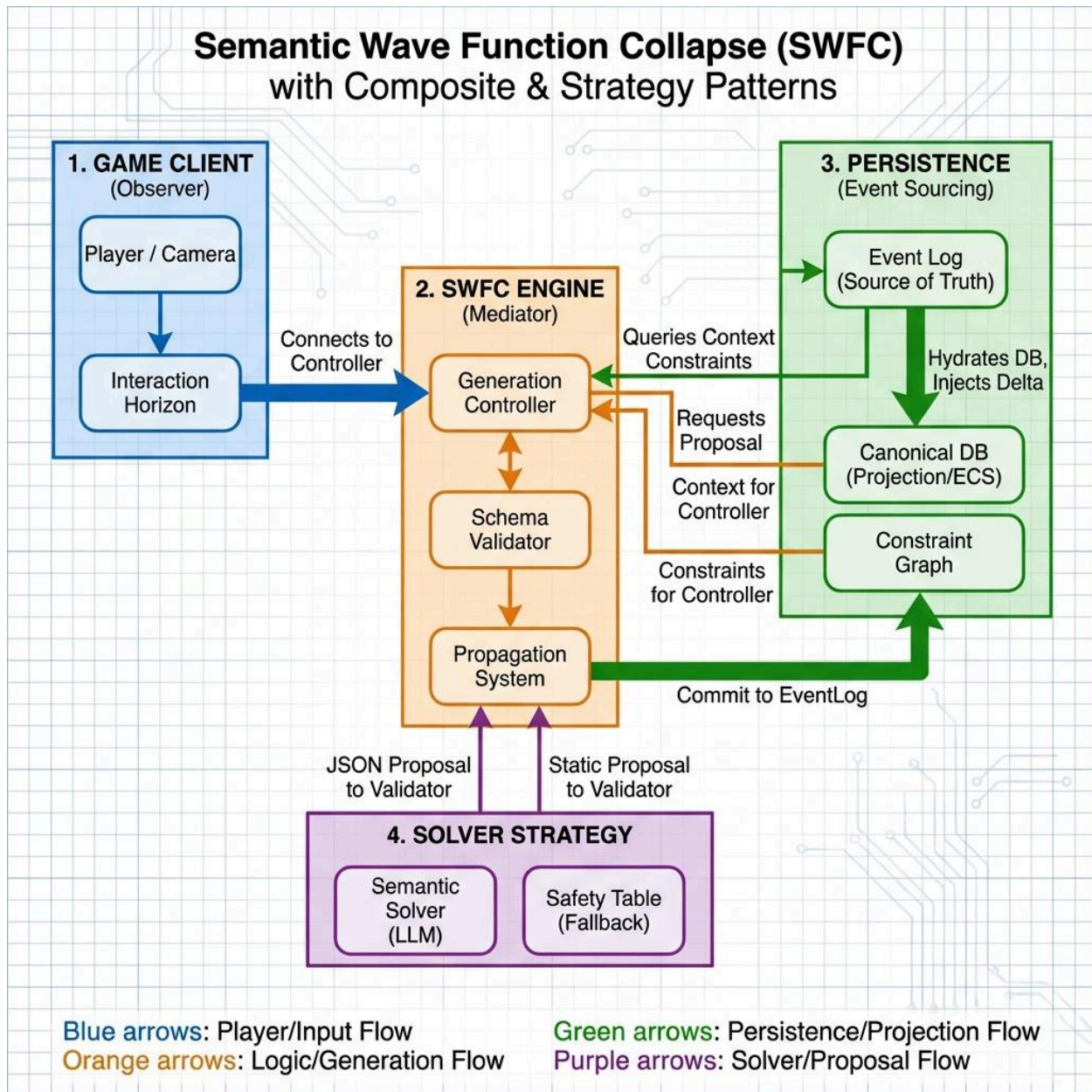
3.3 The "No-Retcon" Invariant

Invariant: Once an entity collapses, its realized components become canonical and may only change through recorded Deltas caused by simulation or player action. The semantic model may propose, but the engine validates and commits. Unobserved entities remain latent, defined by constraints and provenance.

4. Architecture & Data Contracts

4.1 System Architecture

The Engine acts as a **Mediator** to enforce the Strategy and Event Sourcing patterns.



4.2 The Constraint Store (The Constraint Graph)

Instead of a "random seed," unvisited worlds are defined by a **Constraint Graph**.

4.2.1 Constraint Record

Each constraint is stored as a record:

{ key, value, strength (0.0-1.0), type (hard|soft), source_event_id, ttl? }

Pruning Policy:

- Soft constraints are dropped when strength < Strength_Threshold (default 0.15) or when ttl expires.
- Hard constraints persist unless explicitly terminated by ConstraintObsoleted or ConstraintContradicted.

4.3 The Solver Interface (The Contract)

The boundary between the Game Engine and the LLM is strict.

1. **Engine Request:** Sends Context + Constraints + Schema + **Allowed IDs** (Whitelist).
2. **LLM Proposal:** Returns structured JSON (Proposals).
3. **Engine Validation:** Checks schema, ranges, and game rules.
4. **Commit:** Validated data is written to the Event Log.

4.3.1 Deterministic Fallback Protocol

If the LLM fails to produce a valid proposal within N retries, the system executes a

Deterministic Fallback:

- **Selection Mechanism:** Index = Hash(Entity_ID + Hard_Constraints + Ruleset_Version) % Safety_Table_Size.
- **Invariant:** The game loop must never block or crash due to Solver failure.

4.4 Persistence Model (Event Sourcing)

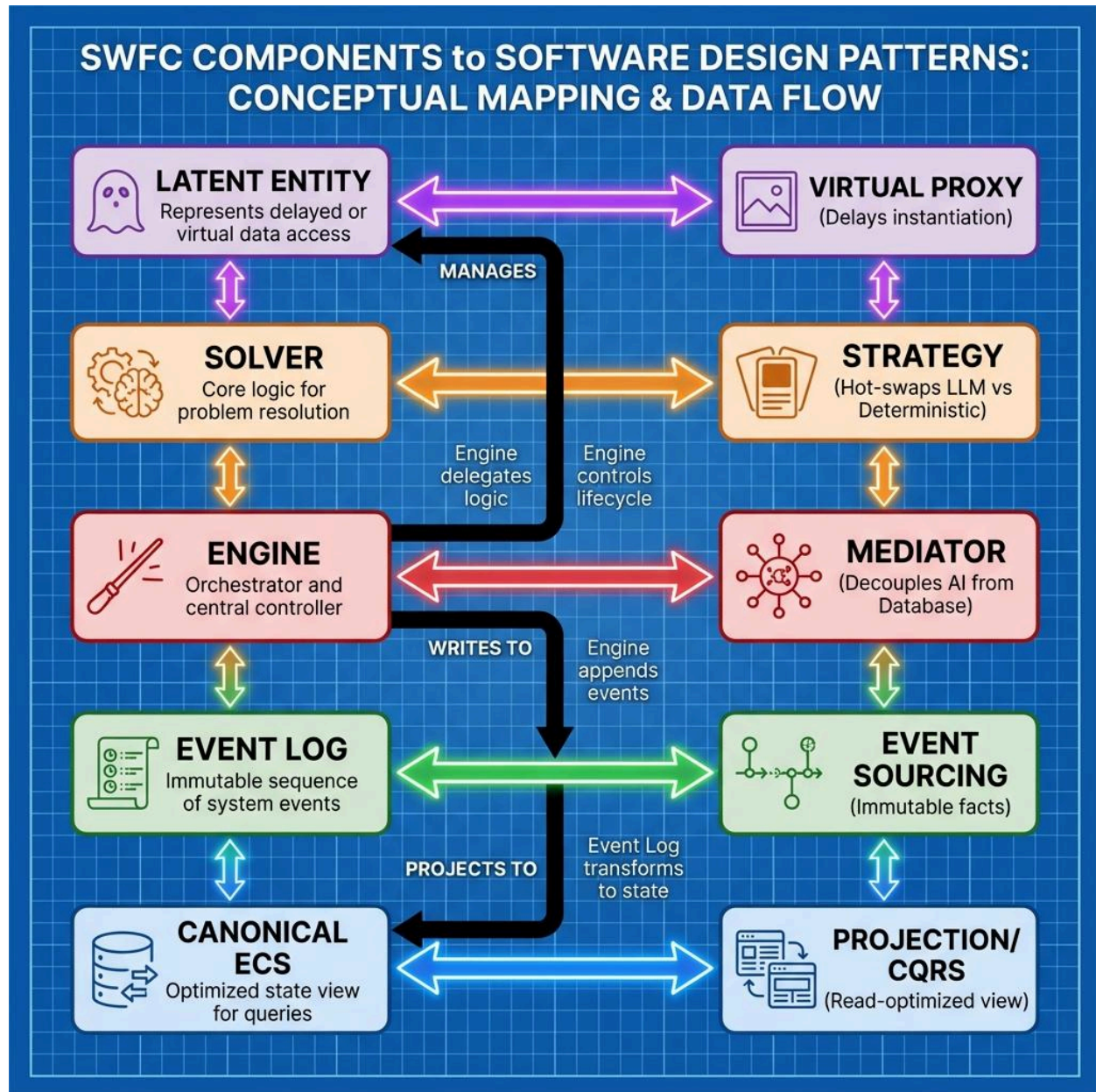
SWFC relies on **Event Sourcing** for replayability and consistency. The **Event Log** is the Source of Truth; the Game World is merely its current projection.

4.4.1 Event Schema

- **CollapseCommitted:** Writes the full component set.
 - *Payload:* { components: { ... }, frozen_stats: { ... }, tags: [...] }
- **DeltaApplied:** Writes patch operations (add/remove/set).
 - *Payload:* { op: "set", path: "/components/Inventory/slots/main/durability", value: 12 }

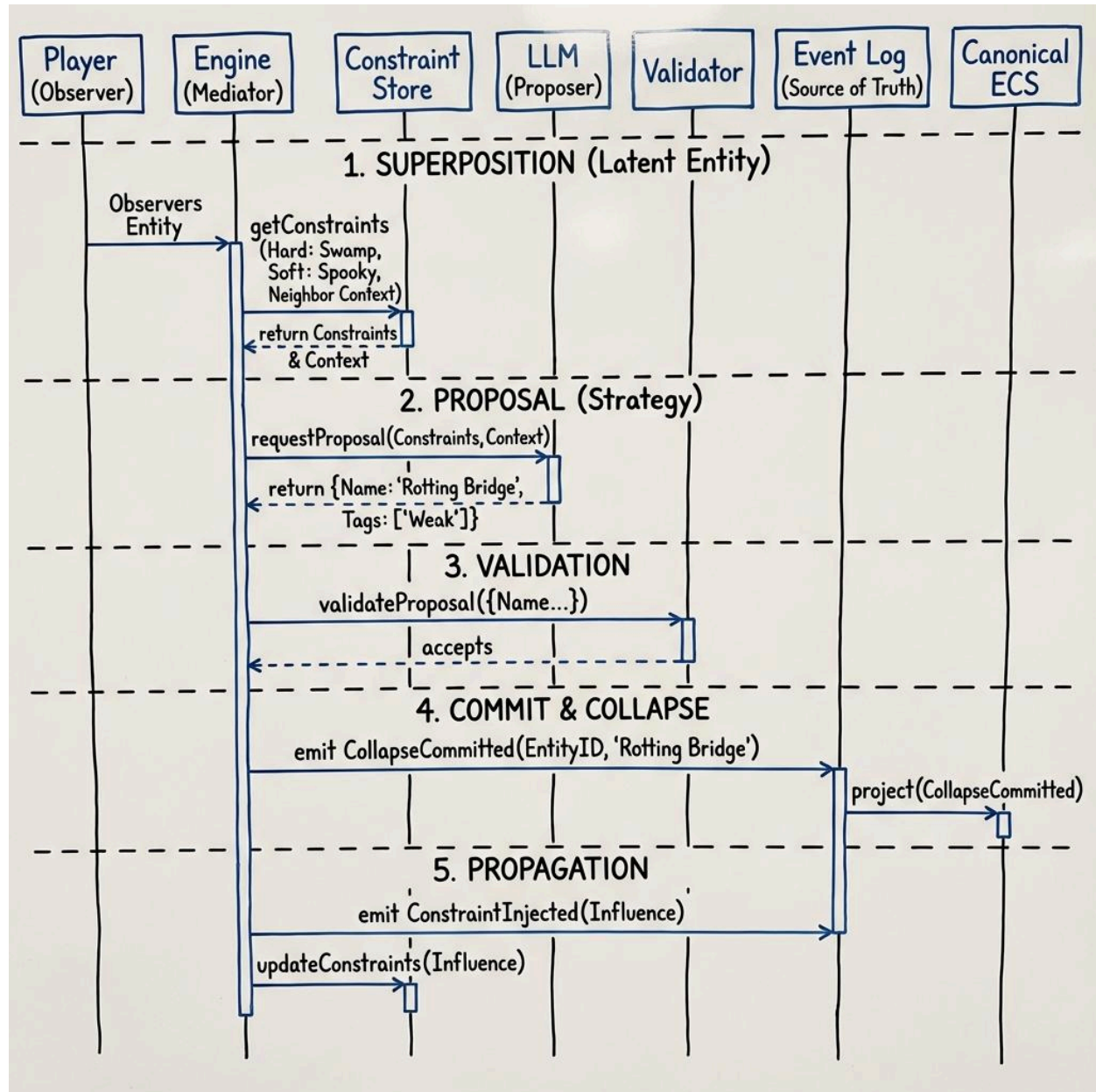
4.5 Architectural Pattern Mapping

SWFC is a composite architecture built on standard software engineering patterns. This mapping ensures the system is maintainable and testable.



5. The SWFC Lifecycle (The Loop)

The following sequence details the flow of data from Observation to Propagation, highlighting the strict separation between the Proposer (LLM) and the Truth (Event Log).



Phase 1: Superposition (Latent State)

The world contains Entity_ID_104.

- State: Latent
- Constraints: { Neighbor: "Industrial_Zone", Global_Event: "Acid_Rain" }

Phase 2: Observation (Trigger)

The player enters the sector containing Entity_ID_104.

Phase 3: Proposal (Semantic Solving)

Engine query: "Propose a sector compatible with Industrial/Acid Rain context."

LLM output: { Type: "Chemical_Runoff_Plant", Hazards: ["Corrosive_Puddles"] }

Phase 4: Validation & Collapse

Engine validates output against ruleset + whitelists.

- *Commit*: CollapseCommitted event written to log
- *Materialize*: Entity_ID_104 projected as Chemical Runoff Plant

Phase 5: Propagation (Constraint Injection)

Engine injects constraints into neighbors via events.

- *Action*: Injects ConstraintInjected event for Entity_ID_105.
- *Constraint*: Ground_Water: Contaminated.

Note on Latency & Future Proofing

The SWFC architecture is **Latency Agnostic**. It utilizes **Lookahead Buffers** (solving Depth+1 while the player is at Depth 0) to mask current LLM inference times. However, the architecture is designed to scale directly with model advances. As inference costs approach zero, the buffer size decreases, but the architectural invariants (Validation, Event Sourcing) remain identical.

6. Conflict Resolution & Precedence

When generating new content, constraints may conflict. The Solver follows this precedence hierarchy:

1. **Canonical State (Highest)**: If a neighbor is already collapsed, its reality is absolute.
2. **Hard Constraints**: Quest requirements or previous Propagation tags.
3. **Soft Constraints**: General themes.
4. **LLM Generation (Lowest)**: The model's "creative filler."

Conflict Policy: If a Hard Constraint violates Canonical State, the system marks the Constraint as "Obsolete" or "False Rumor" rather than altering the map (Retconning).

7. Application Domains (Unified)

7.1 Progressive Mechanical Collapse (Loot)

- **Drop**: Entity is Latent. Constraint: Origin: Swamp_Beast.
- **Inspect**: Collapse Visuals. *Result*: "Slime-coated Blade." (Tags: Toxic, Crude).
- **Equip**: Collapse Stats.

7.1.1 The Translation Layer (Tag-to-Math)

The LLM is a **Selector**, not a Calculator. It generates Semantic Tags which map to Game Math via a **Translation Table**.

- **Tag Canonicalization:** The LLM may only output tags from a provided whitelist.
- **No-Retcon Fix (Balance Patches):** Numeric baseline stats are **Frozen** at the moment of Collapse.
- **The Maintenance Trade-off:** This approach deliberately trades **Runtime Chaos** for **Design-Time Structure**. Maintaining the Translation Table requires defining valid mappings for all tags. This front-loaded effort is the cost of ensuring that high-variance AI outputs result in balanced, bug-free gameplay.

7.2 Reverse Propagation (Cognition)

- **Dialogue:** NPC generates a rumor: "The sewers are flooded."
- **Propagation:** The Engine injects State: Flooded as a Hard Constraint into the unvisited Sewer Sector.
- **Result:** When the player eventually reaches the sewers, the map generator *must* generate water obstacles to satisfy the constraint.

8. Conclusion

SWFC is not merely "LLMs writing text." It is a rigorous system architecture where:

1. **Latent Entities** minimize resource usage.
2. **Event Sourcing** ensures deterministic replay and traceability.
3. **Canonical Projections** prevent "dream logic."
4. **Validation Layers** guarantee engine stability.

This standard provides the blueprint for infinite, coherent worlds that respect player agency and logical cause-and-effect.

9. Appendix: Reference Data Contracts

To facilitate implementation, the following JSON schemas define the core communication protocol between the Engine and the Semantic Solver.

9.1 Solver Request (Engine -> LLM)

```
{
  "request_id": "req_8821a",
  "task_type": "COLLAPSE_ZONE",
  "context": {
    "entity_id": "grid_4_5",
    "global_state": { "weather": "acid_rain", "alert": "high" },
    "neighbors": {
      "north": { "id": "grid_4_4", "tags": ["industrial", "power_plant"] },
      "west": { "id": "grid_3_5", "tags": ["slums"] }
    }
  }
},
```

```

"constraints": {
  "hard": ["must_have_power_access"],
  "soft": ["vibe_decay", "high_crime"]
},
"whitelist": {
  "zone_ids": ["factory", "warehouse", "checkpoint", "clinic"],
  "hazard_ids": ["radiation", "toxin", "fire"]
}
}

```

9.2 Solver Proposal (LLM -> Engine)

```

{
  "request_id": "req_8821a",
  "proposal": {
    "zone_id": "checkpoint",
    "visual_description": "A rusted security gate checking for contamination.",
    "hazards": ["toxin"],
    "tags": ["military", "decay"],
    "reasoning": "Checkpoint fits between Slums and Power Plant; decay fits weather."
  }
}

```

9.3 Validator Response (Engine Internal)

```

{
  "status": "PASS",
  "validated_payload": {
    "zone_id": "checkpoint",
    "components": {
      "Zone": "checkpoint",
      "HazardVolume": "toxin",
      "Visuals": { "skybox": "acid_green", "props": ["barricade"] }
    }
  }
}

```